```java
public class ListaComArray {
    private Integer[] array;
    private boolean resizable;
    private int initialCapacity;
    private int counter;
    private final static int x = 10;
```

array.

É o vetor interno onde os elementos (Integer) são armazenados. Ele funciona como a “base” da lista.

Resizable.

Indica se a lista pode ou não ser redimensionada quando ficar cheia.

initialCapacity.

Define a capacidade inicial do array, ou seja, quantos elementos cabem logo na criação da lista.

Conter.

Guarda a quantidade de elementos atualmente armazenados.

x.

É uma constante estática com valor fixo 10.

*Usada como incremento quando a Lista preciso ser redimensionada (resizeArrayList).

*Não muda durante a execução do programa.

COMO ELES SE RELACIONAM:

*Quando você cria a lista, o array recebe o tamanho initialCapacity.

*Cada vez que você adiciona um elemento, conter aumenta.

*Se conter chegar ao limite (array.length), o método resizeArrayList() cria um novo array maior, somando x posições.

*O atriburo rezable decide se esse redimensionamento pode acontecer ou não.

EM RESUMO:

Array : Onde os dados ficam.

Conter: Controla quantos dados já foram inseridos.

InitialCapacity: Define o tamanho inicial.

Resizable: Decido se pode crescer.

X: define de quanto em quanto cresce.

CONSTRUTORES:

```
public ListaComArray() {
    this(10);
}
```

Construtor padrão> Cria a lista com capacidade inicial 10 e redimensionável por padrão. É útil quando você não quer se preocupar com parâmetos.

```
public ListaComArray(int initialCapacity) {
    this(initialCapacity, true);
}
```

Permite escolher a capacidade inicial, mas assume que a lista é redimensinável. Ex: new ListaComArray(50) cria uma lista com espaço para 50 elementos.

```
public ListaComArray(int initialCapacity, boolean resizable) {
    this.initialCapacity = initialCapacity;
    this.resizable = resizable;
    this.counter = 0;
    this.array = new Integer[initialCapacity];
}
```

Dá controle total: você define a capacidade inicial es se alista pode ou não crescer. Ex:new ListaComArray(20, false) cria uma lista fixa de 20 posições.

POR QUE É UTIL

*flexibilidade: você pode criar listas pequenas, grandes, fixas ou expansíveis conforme a necessidade.

*Reuso de códifo: os construtores chamam uns aos outros (this(...)), evitando duplicação de lófica.

*Boas práticas: facilita para quem usa a classe, oferecendo opções simples (padrão) e avançadas (com parâmetros).

METODOS

Método add(Integer obj)

Este método é o coração da sua lista, ele foi implementado para inserir um novo elemento no final da estrutura.

```
public boolean add(Integer obj){
    if(counter >= array.length){          // 1. Verifica se o array
está cheio
        if(!resizable){                   // 2. Se não puder
redimensionar
            return false;                 //    retorna false (não
adiciona)
        }
        resizeArrayList();                // 3. Se puder, aumenta a
capacidade
    }
```

```
    array[counter] = obj;                   // 4. Insere o elemento na
posição livre
    counter++;                              // 5. Atualiza o contador
    return true;                            // 6. Confirma que adicionou
}
```

PASSO A PASSO

*Verificação de espaço: compara conter com array.length.

*Controle de redimensionamento: se resizable for false, não há como crescer e o método retorna false.

*Inserção: coloca o novo objeto na posição conter.

*Atualização: incrementa o contador para refletir o novo tamanho lógico da lista.

*Retorno: devolve true para indicar sucesso.

Add(int index, Integer element)

Esse método é a versão que permite inserir um elemento em uma posição específica da lista, não apenas no final.

```
public boolean add(int index, Integer element){
    if(index < 0 || index > counter){      // 1. Valida o índice
        return false;
    }
    if(counter == array.length){           // 2. Verifica se o array
está cheio
        if(resizable){                     // 3. Se puder redimensionar
            resizeArrayList();             //    aumenta a capacidade
        }else{
            return false;                  // 4. Se não puder, falha
        }
    }

    // 5. Desloca os elementos para a direita
    for(int i = counter; i > index; i--){
        array[i] = array[i - 1];
    }

    // 6. Insere o novo elemento na posição desejada
    array[index] = element;

    // 7. Atualiza o contador
    counter++;

    return true;                           // 8. Confirma sucesso
}
```

PASSO A PASSO

*Validação: garante que o índice é válido (não negativo e não maior que conter).

*Checagem de capacidade: so o array estiver cheio, decide se pode crescer (resizable).

* reciszeArrayList: aumenta o tamanho do array se necessário.

*Deslocamento: move todos os elementos a partir do índice para a direita, abrindo espaço.

*Inserção: coloca o novo elemento na posição correta.

*Atualização: incrementa conter para refletir o novo tamanho lógico da lista.

*Retorno: devolve true se conseguiu iserir.

resizeArrayList()

Este método pe responsável por aumentar acapacidade da lista quando ela fica cheia. Ele pe chamado automaticamente dentro dos métodos add quando conter == array.length e resizable == true.

```
private void resizeArrayList(){
    // 1. Cria um novo array maior, somando 'x' posições
    Integer novo[] = new Integer[array.length + x];

    // 2. Copia todos os elementos do array antigo para o novo
    System.arraycopy(array, 0, novo, 0, counter);

    // 3. Substitui o array antigo pelo novo
    array = novo;
}
```

PASSO A PASSO

*Novo array: cria um array maior que o atual, adicionado x posições extras.

*Copia de elementos: usa System.arraycopy para transferir todos válidos (conter) para o novo array.

*Atualização da referência: substitui o array antigo pelo novo, grantindo que a lista agora tenha mais espaço.

removeFirst(Integer elemento)

Esse método foi implementado para remover a primeira ocorrência de um elemento específico na lista.

```
public boolean removeFirst(Integer element){
    int index = indexOff(element);      // 1. Busca o índice do
elemento
    if(index == -1){                    // 2. Se não encontrou
        return false;                   //    retorna false
    }
    remove(index);                      // 3. Remove o elemento na
posição encontrada
    return true;                        // 4. Confirma que removeu
```

```
}
```

PASSO A PASSO:

*indexOff: procura o índice da primeira ocorrência do elemento. Se não encontrar, retorna -1.

*Validação: se o índice for -1, significa que o elemnto não existe na lista – retirba flase.

*remove: chama o método remove(int index), que desloca os elementos para a esquerda e ajusta o conter.

*Retorno: devolve true para indicar que a rmoção foi feita com sucesso.

Em resumo esse método é um atalho para remover um elemento pelo valor, sem precisar saber o índice. Ele combina a busca (indexOff) com a remoção (remove).

Public integer get(int index)

***Função: retorna o elemento armazenado na posição indicada.

*Validação: se o índice for inválido (menor que 0 ou maior/igual ao número de elementos). Retorna null.

*Uso típico: acessa elementos sem alterar nada na lista.

```
public Integer get(int index){
    if(index < 0 || index >= counter){
        return null;
    }
    return array[index];
}
```

Public void clear()

***Função: limpa a listam removendo todos os elementos.

*Se for redimensionável: recria o array com a capacidade inicial.

*Se não for redimensionável: apenas zera o contador, mas mantém o array existente.

*Uso típico: resetar alista para começar do zero.

```
public void clear(){
    if (resizable){
        array = new Integer[initialCapacity];
    }
    counter = 0;
}
```

Public int size()

*Função: retorna quantos elementos estão atualmente armazenados.

*Uso típico: Saber o tamanho lógico da lista (não confundir com array,length, que é a capacidade).

Public integer set(int index, Integer elemento)

Este método é bem direto, ele serve para substituir o valor que já existe em uma posição específica da lista por um novo valor, e ao mesmo tempo devolver o valor antigo que estava lá.

```
public Integer set(int index, Integer element){
    if(index < 0 || index >= counter){   // 1. Valida o índice
        return null;                     //    se inválido, retorna
null
    }
    Integer oldValue = array[index];     // 2. Guarda o valor antigo
    array[index] = element;              // 3. Substitui pelo novo
valor
    return oldValue;                     // 4. Retorna o valor antigo
}
```

PASSO A PASSO

***Valoidação de índice: garante que você não está tentando acessar fora dos limites da lista.

*oldValue: captura o valor que estava na posição antes da troca.

*Sustituição: coloca o novo elemento no índice indicado.

*Retorno: devolve o valor antigo, útil para saber o que foi sobrescrito.

Em resumo: esse método é usado quando você quer atualizar um elemento já existente na lista. Ele não aumenta nem diminui o tamanho (conter continua igual), apenas troca valores.

Public boolean isEmpty()

Este método verifica se a lista está vazia, se não há nenhum elemento armazenado

```
public boolean isEmpty(){
    return (counter == 0);
}
```

COMO FUNCIONA

*Counter guarda quantos elementos foram adicionados.

*Se conter == 0, significa que nenhum elemento foi inserido – lista vazia.

*O método retorna true nesse caso.

*Caso contrário, retorna false.

EM RESUMO

*isEmpty() é um método utilizado que facilita a checagem rápida do estado da lista

*Ele éisolado, não altera nada, apenas consulta o valor de conter.

*É muito usado em conjunto com outros métodos, por exemplo, antes de chamar remove() ou get() para evitar erros.

Public boolean isFull()

Este método verifica se alista está cheia, ou seja, se não há mais espaço para adicionar novos elementos sem redimensionar:

```
public boolean isFull(){
    if(!resizable){                          // 1. Só faz sentido se
não puder crescer
        return (counter == array.length);    // 2. Checa se o contador
atingiu o limite
    }
    return false;                            // 3. Se for
redimensionável, nunca estará "cheia"
}
```

PASSO A PASSO

*Verificação de redimensionamento: se resizable for falsem a lista tem tamanho fixo.

*Comparação: compara conter (quantos elementos foram adicionados) com array.length (capacidade total).

*Retorno:

*true> lista está cheia e não pode receber mais elementos.

*false > alinda há espaço ou a lista é expansível.

EM RESUMO

***isEmpty() verifica se não há elementos.

*isFull**()** verifica se não há espaço para novos elementos (apenas em listas fixas).

*junto, eles permitem controlar o estado da lista antes de operações com add() ou remove()

Public int constains(integer elemento)

Este método tem a função de verificar se alista está cheia, mas com uma condição importante: ele só considera a lista "cheia" se ela não for redimensionável.

```
public boolean isFull(){
```

```
    if(!resizable){
        return(counter == array.length);
    }
    return false;
}
```

O QUE ELA FAZ:

***CHECA RESIZABLE:**

*Se resizable for false, significa que alista tem tamanho fixo.

*Nesse caso, compara conter (quatnos elementos já foram adicionados) com array.length (capacidade máxima).

*Se forem iguais, retorna true – lista cheia.

*__Se__ resizable __for__ true:

*O método retorna sempre false, porque mesmo que o array esteja cheiom ele pode crescer com resizeArrayList().

*Ou seja, uma lista expansível nunca pe considerada "cheia".

EM RESUMO

***Para listas fixas, isFull() indica corretamente quando não hã mais espaço.

*Para listas expansíveis, ele sempre retorna false, já que o espaço pode ser aumentado.

Public int indexOf(integer element)

Este método é o responsável por encontrar a posição da primeira ocorrência de um elemento na lista.

```
public int indexOf(Integer element){
    for(int i = 0; i < counter; i++){
        if(array[i] != null && array[i].equals(element)){
            return i;
        }
    }
    return -1;
}
```

O QUE ELE FAZ:

*Percorre o array: vai do índice 0 até conter -1, ou sejam apenas sobre os elementos válidos da lista.

*Verifica null: garnate que não vai dar erro de NullPointrExceprion ao chamar .equals.

*Compara elementos: se o valor encontrado for igual ao procurado, retorna o índice imediatamente.

*Retorno padrão: se não encontrar o elemento, retorna -1, que é a convenção em java para "não encontrado".

EM RESUMO:

***Esse método é usado para localizar a primeira ocorrência de um valor.

*Ele é útil em conjunto com métodos como removeFirst ou contains.

*Se o elemento não existir, retorna -1

public int lastIndexOff(Integer elemento)

Eate método foi para encontrar a última ocorrência de um elemento na lista.

```
public int lastIndexOf(Integer element){
    for(int i = counter - 1; i >= 0; i--){
        if(array[i] != null && array[i].equals(element)){
            return i;
        }
    }
    return -1;
}
```

O QUE ELE FAZ:

*Percorre o array de trás para frente (do último elemento válido até o primeiro).

*Se encontrar o elemento, retorna o índice da última ocorrência.

*Se não encontrar, retorna -1.

Public[] integer toArray[]

Este método tem a função de criar uma cópia externa da lista em forma de array, contendo apenas os elementos válidos já adicionados.

```
public Integer[] toArray(){
    // Cria um novo array com o tamanho exato dos elementos já
adicionados
    Integer[] novoArray = new Integer[counter];
    // Copia os elementos do array interno para o novo array
    System.arraycopy(array, 0, novoArray, 0, counter);
    // Retorna o novo array
    return novoArray;
}
```

O QUE ELE FAZ

***Novo array: cria um attay com tamanho igual ao número de elementos (conter), não à capacidade total (array.length).

*Cópia: usa System.arraycopy para o novo.

*Retorno: devolve esse novo array, que é independente do array interno da classe.

EM RESUMO:

*****O método serve para exportar os dados da lista sem expor diretamente o array interno.

*Isso evita que quem usa a classe altere os elementos internos sem passar pelos métodos de controle (add, remove, etc).

*É muito útil quando você quer trabalhar com os elementos em outro contexto, como imprimir ordenar ou passar para outra função.

Public String toString()

Método que retorna uma representação textual da lista, mostrando tanto os elementos já adicionados quanto o array interno completo (incluindo os espaços vazios).

```
public String toString(){
    String myarray1 = "[ ";
    for(int i = 0; i < counter; i++){
        if(i != (counter-1)){
            myarray1 += array[i] + ", ";
        }else{
            myarray1 += array[i] + " ]";
        }
    }

    String myarray2 = "[";
    for(int i =0; i < array.length; i++){
        if(i != (array.length-1)){
            myarray2 += array[i] + ", ";
        }else{
            myarray2 += array[i] + " ]";
        }
    }

    return "@@@\n" + myarray1 + "\n" + myarray2;
}
```

O QUE ELE FAZ:

*****myarray1: monta uma string com apenas os elementos válidos (até conter).

*muattay2: monta uma string com todo o array interno (array.length), incluindo posições ainda não usadas (que podem ser null)

*Retorno: junta as duas representações em uma única stringm separadas por uma quebra de linha e prefixadas por “@@@”.

EM RESUMO

*****Esse toString() é útil para debug: você consegue ver ao mesmo tempo o estado lógico da lista (quantos elementos foram adicionados) e o estado físico do array interno (capacidade e espações vazios).

*É diferente do toArray(), que retorna apenas os elementos válidos em um novo array.

OU SEJA:

*A primeira linha mostra apenas os elementos adicionados.

*A segunda linha mostra o array interno completo, com os null indicando espaços ainda livres.